

BuildStart Software Engineering Intern

2-Day Engineering Challenge

Automated Bank Payment Verification System

Time limit: 2 days

Objective: Build a reliable, low cost system that can determine whether a customer's bank transfer payment can be safely accepted from the evidence available to the business.

1. The Problem

BuildStart creates WhatsApp Agents for businesses.

A typical payment flow is:

1. A customer places an order through WhatsApp.
2. The Agent provides the business's bank details.
3. The customer makes a bank transfer.
4. The customer sends a screenshot or photograph of the payment slip.
5. The bot must determine whether the payment is valid and belongs to that order.

There is **no bank API available** for this challenge.

Your system must therefore work using available evidence such as:

- Payment slip images
- Information extracted from those images
- order information
- customer information
- bank SMS notifications received by the business
- previous payment records

You decide how these sources should be used.

2. Your Task

Build a working prototype that can process a payment submission and produce one of three decisions:

APPROVED

There is sufficient evidence to safely accept the payment.

REJECTED

There is sufficient evidence that the payment is invalid, incorrect, already used, or fraudulent.

NEEDS VERIFICATION

There is not enough reliable evidence to safely approve or reject it.

For every decision, the system should provide a clear reason.

The system should also determine what should happen next.

For example:

Payment amount does not match the order.
Please send the correct payment slip.

or:

The payment appears valid, but the transaction cannot currently be matched with a bank notification.
Please wait or contact the business.

3. What Your System Must Handle

Your solution should consider at least the following situations.

Normal payment

A genuine payment with matching information.

Wrong amount

The customer paid less or more than the required amount.

Wrong account

The payment was made to an account that does not belong to the business.

Duplicate payment

The same payment slip is submitted more than once.

Reused payment

A genuine payment is submitted for another customer's order.

Same payment, different image

The same transaction is submitted as different screenshots/photos/crops.

Old payment

A genuine payment from a previous transaction is submitted for a new order.

Edited or suspicious slip

Information such as the amount, date, reference, or account details may have been manipulated.

Unclear image

The slip is blurry, cropped, dark, low-resolution, or otherwise difficult to read.

Conflicting evidence

Information extracted from the slip does not agree with available bank SMS or order information.

Missing evidence

The system cannot confidently establish that the payment occurred.

Multiple customers with similar payments

Several customers may pay the same amount around the same time.

Your system must avoid assuming that matching amounts alone prove that a payment belongs to a particular customer.

4. Bank SMS

Businesses may receive transaction notifications through SMS.

For example:

Rs. 25,000 credited to A/C XXXX1234. Ref 839201.

Your system may use these messages as additional evidence. Find a reliable way to get that message to your implemented system

Consider that:

- SMS may arrive later than the slip.
- SMS may not exist.
- SMS formats may differ.
- Multiple payments may have the same amount.
- A transaction may need to be matched using several fields.

You decide how much trust to place in each source of evidence.

5. Fraud Detection

Fraud prevention is a major part of this challenge.

A payment slip should **not** be considered genuine simply because it visually resembles a bank slip.

Think about how your system could identify:

- reused payments
- duplicate submissions
- manipulated images
- inconsistent transaction information
- old transactions
- suspicious references
- mismatched accounts
- mismatched dates
- mismatched amounts
- conflicting evidence
- attempts to associate another customer's payment with an order

You are expected to discover additional fraud scenarios yourself.

6. Cost Requirement

The system must achieve the **lowest practical cost while maintaining high accuracy**.

Do not simply send every image and every piece of information to an expensive AI model.

Think about:

- when AI is actually necessary
- cheaper alternatives
- avoiding unnecessary processing
- duplicate detection
- caching
- escalation
- model selection
- processing only difficult cases
- API usage
- Server cost
- latency

The cheapest solution is not automatically the best solution.

If spending more produces a meaningful improvement in payment safety, explain why that cost is justified.

7. UI/UX Requirement

This is not only a backend/AI challenge.

Build a **clean business-facing interface** for reviewing payment submissions.

A business employee should be able to understand a payment quickly and answer:

"Can I safely accept this payment?"

The interface should make important information immediately visible, including where relevant:

- customer
- order
- expected amount
- submitted amount
- payment date
- transaction/reference number
- receiving account
- verification result

- confidence/uncertainty
- reasons
- suspicious indicators
- related or duplicate payments
- bank SMS evidence
- submitted payment image

The interface should also clearly distinguish between:

What the customer submitted

and

What the system concluded.

Think carefully about information hierarchy, interaction design, and the workflow of a real business employee.

You decide what additional UI features are useful.

8. Customer Experience

When a payment cannot be safely verified, the WhatsApp bot should be able to respond appropriately.

Examples:

We couldn't clearly read the payment slip. Please send a clearer image of the complete slip.

The payment amount does not match your order. Please check the payment and send the correct slip.

This payment appears to have already been used for another order. Please send the correct payment slip.

The system should **avoid exposing unnecessary internal fraud detection information** to the customer.

You decide what the customer should see and what should remain internal.

9. Testing

Your testing should include both normal and problematic cases.

You should identify:

- false approvals
- false rejections
- incorrect matches
- cases requiring manual verification
- unexpected behaviour

False approval is particularly important.

A system that incorrectly approves a fraudulent payment can cause direct financial loss.

10. Debugging

Your first implementation is not expected to be perfect.

Document at least **three meaningful problems** you encountered.

For each:

- What failed?
- Why did it fail?
- How did you identify the cause?
- What did you change?
- Did the change improve the result?

We are interested in your debugging methodology, not just the final result.

11. Technology

There are **no prescribed technologies**.

You may choose any:

- programming language
- framework
- database
- AI model
- OCR system

- Image processing approach
- API
- local model
- cloud service

If you encounter a technology you have never used before, research it and use it if appropriate.

The solution is open ended.

We are evaluating your engineering decisions rather than your knowledge of a particular technology stack.

12. AI-Assisted Development

You are **allowed and encouraged to use AI coding tools**.

You may use tools such as:

- ChatGPT
- Claude
- Gemini
- Cursor
- GitHub Copilot
- Lovable
- Antigravity
- other AI development tools

Using AI is not a disadvantage.

However, **you are responsible for the final implementation**.

You should understand and be able to explain:

- what the system does
- why you designed it this way
- what the important components do
- why you selected your approach
- how you handled failures
- how you controlled cost

During the evaluation, we may ask you to modify, debug, or extend your implementation.

13. Deliverables

1. Working Prototype

A deployed application is preferred.

Otherwise, provide clear local setup instructions.

2. Source Code

Provide the complete source code through GitHub or another accessible repository.

3. README

Include:

- setup instructions
- architecture overview
- major design decisions
- verification approach
- fraud-handling approach
- cost considerations
- known limitations

4. Short Engineering Report

Approximately **2-3 pages** covering:

1. How your solution works.
2. How you verify a payment belongs to an order.
3. How you detect duplicate/reused payments.
4. How you handle suspicious or manipulated slips.
5. How you use bank SMS information.
6. How you minimise cost.
7. Your major limitations.
8. What you would change for production scale.

5. Demonstration

Provide a short video or live demonstration showing several different payment scenarios, including:

- valid payment
- wrong amount
- duplicate/reused payment
- suspicious payment
- unclear slip
- insufficient evidence

14. Evaluation

We will evaluate the submission approximately as follows:

Area	Weight
Problem solving & engineering judgement	25%
Payment accuracy & fraud handling	25%
Cost & performance	15%
Engineering quality & reliability	15%
Debugging & testing	10%
UI/UX engineering	10%

Problem Solving & Engineering Judgement

Can you understand an ambiguous real-world problem and design an effective solution without being given the architecture?

Accuracy & Fraud Handling

Can the system safely distinguish legitimate, invalid, duplicate, reused, suspicious, and uncertain payments?

Cost & Performance

Can you achieve strong results without unnecessary AI/API usage?

Engineering Quality

Is the system structured, reliable, maintainable, and resilient to failures?

Debugging

Can you systematically find and fix problems?

UI/UX

Can a real business employee understand and act on the verification result quickly?

15. What We Care About Most

There is **no single correct solution**.

We are looking for candidates who can:

- identify problems that were not explicitly mentioned
- research unfamiliar technologies quickly
- make sensible technical trade-offs
- build rather than over-engineer
- debug systematically
- think about security and fraud
- optimise cost without blindly sacrificing accuracy
- design interfaces around real users
- recognise uncertainty instead of pretending the system is always correct

A sophisticated architecture is not automatically a good solution.

A simple solution with strong reasoning, good accuracy, low cost, and excellent failure handling can be significantly better.

16. Production Thinking

Finally, assume your prototype succeeds.

BuildStart now needs to process **100,000+ payment submissions per month** across multiple businesses.

Consider how your solution would need to evolve as:

- transaction volume increases
- fraud attempts increase
- different banks are introduced
- SMS formats differ
- image quality varies
- AI/API costs increase
- external services fail
- more cases require human review

You do not need to implement this.

Explain what you would change and why.

The Objective

Build the **most reliable payment-verification system you can within two days**, while maintaining a strong balance between:

Accuracy + Fraud Prevention + Cost + Speed + User Experience

The implementation is up to you.

Good luck!